

mrpFeedback

1 Executive Summary

This module provides a general MRP feedback control law. This 3-axis control can asymptotically track a general reference attitude trajectory. The module is setup to work with or without N reaction wheels with general orientation. If the reaction wheel states are fed into this module, then the resulting RW gyroscopic terms are compensated for. If the wheel information is not present, then these terms are ignored.

2 Message Connection Descriptions

The following table lists all the module input and output messages. The module msg variable name is set by the user from python. The msg type contains a link to the message structure definition, while the description provides information on what this message is used for.

Table 1: Module I/O Messages

Msg Variable Name	Msg Type	Description
cmdTorqueOutMsg	CmdTorqueBodyMsgPayload	Control torque output message
intFeedbackTorqueOutMsg	CmdTorqueBodyMsgPayload	Integral feedback control torque output message
guidInMsg	AttGuidMsgPayload	Attitude guidance input message
vehConfigInMsg	VehicleConfigMsgPayload	Vehicle configuration input message
rwParamsInMsg	RWArrayConfigMsgPayload	Reaction wheel array configuration input message
rwSpeedsInMsg	RWSpeedMsgPayload	Reaction wheel speeds message
rwAvailInMsg	RWAvailabilityMsgPayload	Reaction wheel availability message.

3 Module Parameters

The following table lists all the module parameters than can be set. The parameters are optional unless indicated (if not specified default is used).

Table 2: Module Parameters

Parameter Name	Type	Units	Default
K	float	[rad/s]	0
P	float	[N m s]	0
Ki	float	[N m]	0
integralLimit	float	[N m]	0
controlLawType	enum	[-]	0
	ControlLawType		
knownTorquePntB_B	Eigen::Vector3f	[N m]]	[0, 0, 0]

4 Module Assumptions and Limitations

This module assumes the main spacecraft is a rigid body. If RW devices are installed, their wheel speeds are assumed to be fed into this control solution.

5 Initialization

The module is configured by:

```
module = mrpFeedback.MrpFeedback()
module.modelTag = "mrpFeedback"
module.K = K
module.P = P
module.Ki = Ki
module.integralLimit = integral_limit
module.controlLawType = control_law_type
module.knownTorquePntB_B = known_torque
```

6 Detailed Module Description

6.1 General Function

The `mrpFeedback` module creates the MRP attitude feedback control torque $\mathbf{L}_r$ developed in chapter 8 of Analytical Mechanics of Space Systems. The output message is a body-frame control torque vector. The required attitude guidance message contains both attitude tracking error states as well as reference frame states. This message is read in with every update cycle. The vehicle configuration message is only read in on reset and contains the spacecraft inertia tensor about the vehicle's center of mass location.

The MRP feedback control can compensate for Reaction Wheel (RW) gyroscopic effects as well. This is an optional input message where the RW configuration array message contains the RW spin axis $\hat{\mathbf{g}}_{s,i}$ information and the RW polar inertia about the spin axis $I_{W_{s,i}}$. This is only read in on reset. The RW speed message contains the RW speed Ω_i and is read in every time step. The optional RW availability message can be used to include or not include RWs in the MRP feedback. This allows the module to selectively turn off some RWs. The default is that all RWs are operational and are included.

6.2 Initialization

Simply call the module reset function prior to using this control module. This will reset the prior function call time variable, and reset the attitude error integral measure. The control update period Δt is evaluated automatically.

6.3 Algorithm

This module employs the MRP feedback algorithm of Example (8.14) of Analytical Mechanics of Space Systems. This nonlinear attitude tracking control includes an integral measure of the attitude error. Further, we seek to avoid quadratic ω terms to reduce the likelihood of control saturation during a detumbling phase. Let the new nonlinear feedback control be expressed as

$$[G_s]\mathbf{u}_s = -\mathbf{L}_r$$

where

$$\begin{aligned} \mathbf{L}_r = & -K\sigma - [P]\delta\omega - [P][K_I]\mathbf{z} - [I_{RW}](-\dot{\omega}_r + [\tilde{\omega}]\omega_r) - \mathbf{L} \\ & + ([\tilde{\omega}_r] + [\widetilde{K_I\mathbf{z}}]) ([I_{RW}]\omega + [G_s]\mathbf{h}_s) \end{aligned}$$

and

$$h_{s_i} = I_{W_{s_i}}(\hat{\mathbf{g}}_{s_i}^T \omega_{B/N} + \Omega_i)$$

with I_{W_s} being the RW spin axis inertia.

The integral attitude error measure $\mathbf{z}$ is defined through

$$\mathbf{z} = K \int_{t_0}^t \sigma dt + [I_{RW}] \delta \omega$$

A limit to the magnitude of the $\int_{t_0}^t \sigma dt$ can be specified, which is a scalar compared to each element of the integral term.

The integral measure $\mathbf{z}$ must be computed to determine $[P][K_I]\mathbf{z}$, and the expression $[\widetilde{K_I}\mathbf{z}]$ is added to $[\widetilde{\omega_r}]$ term.

7 User Guide

This module requires the following variables from the required input messages:

- $\sigma_{B/N}$ as `guidCmdData.sigma_BR`
- ${}^B\omega_{B/R}$ as `guidCmdData.omega_BR_B`
- ${}^B\omega_{R/N}$ as `guidCmdData.omega_RN_B`
- ${}^B\dot{\omega}_{R/N}$ as `guidCmdData.domega_RN_B`
- $[I]$, the inertia matrix of the body as `vehicleConfigOut.ISCPntB_B`

The gains K and P must be set to positive values. The integral gain K_i is optional, it is 0 by default, in which case the error integration for the controller is disabled, leaving just PD terms. The integrator is required to maintain asymptotic tracking in the presence of an external disturbing torque. The `integrallimit` is a scalar value applied in an element-wise check to ensure that the value of each element of the $\int_{t_0}^t \sigma dt$ vector is within the desired limit. If not, the sign of that element is persevered, but the magnitude is replaced by `integrallimit`.

If the `rwParamsInMsg` is specified, then the associated `rwSpeedsInMsg` is required as well.

The `rwAvailInMsg` is optional and is used to selectively include RW devices in the control solution.

The `controlLawType` is an input that enables the user to choose between two different control laws. When `controlLawType = NORMAL`, the control law is that specified above. Otherwise, the control law takes the form:

$$\begin{aligned} \mathbf{L}_r = & -K\sigma - [P]\delta\omega - [P][K_I]\mathbf{z} - [I_{RW}](-\dot{\omega}_r + [\tilde{\omega}]\omega_r) - \mathbf{L} \\ & + [\tilde{\omega}]([I_{RW}]\omega + [G_s]\mathbf{h}_s). \end{aligned}$$

This control law is also asymptotically stable. The advantage when compared to `controlLawType = NORMAL` is that in this one, the integral control feedback, which may contain integration errors, only appears once. On the downside, this control law depends quadratically on the angular rates of the spacecraft, and could cause a large control torque when the spacecraft is tumbling at a high rate. When unspecified, this parameter defaults to `controlLawType = NORMAL`.